



DFA vs cDFA for regular counting
using regular constraints with lazy clause evaluation

Daniël Ravensbergen¹

Supervisors: Emir Demirović¹, Imko Marijnissen¹

¹EEMCS, Delft University of Technology, The Netherlands

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
June 05 2026

Name of the student: Daniël Ravensbergen

Final project course: CSE3000 Research Project

Thesis committee: Emir Demirović, Imko Marijnissen, <Examiner>

An electronic version of this thesis is available at <http://repository.tudelft.nl/>.

Abstract

cDFA's offer a more natural encoding of counting regular patterns—a prevalent problem in timetabling and sequencing—than widely-used DFA's. A cDFA-based propagator for finite domain constraint solving has been shown to propagate more than the decomposition of a cDFA-based regular constraint. This paper extends that algorithm with explanations for lazy clause generation and shows that it is more efficient [briefly elaborate how] to model combinatorial problems with cDFA over DFA when [insert conditions of the cDFA].

1 Introduction

Constraint programming (CP) is a paradigm for solving finite domain combinatorial satisfaction and optimization problems. These problems are often NP-Hard, meaning they require backtracking in the search space. CP allows for the pruning of unvisited branches of the search space. It does this by propagating constraints a solution needs to satisfy. One of these constraints is the regular constraint. It is satisfied when its input is in a given regular language.

Regular counting is a feature of regular languages that specifies how many times a pattern is allowed to occur. It is well known as the regular expression operation $\{n\}$. According to Chapman and Stolee[1] regular counting using regex is used in 20% of 1204 python projects[2]. Regular counting is useful for constraining a variety of problems such as sequencing and timetabling[3] where you want to specify the number of occurrences of patterns.

Regular counting can be modeled for CP with the aforementioned regular constraints, which are often based on deterministic finite automaton (DFA). However, Beldiceanu et al.[3], [4] show that a dedicated constraint based on a counter-DFA (cDFA) is not only more straight-forward to model regular counting with, but also more efficient, as it prunes more branches, providing significant speed-up of computation. Even though they provide an algorithm for exact regular counting that is only bounds consistent[3], they show that it prunes more than earlier propagators.

Up to now, this algorithm had not been implemented with explanations for Lazy Clause Generation (LCG)—a newer technology in the field of CP. It was also not clear for what sizes of cDFA it is beneficial to use this constraint over the readily available DFA-based regular constraints.

This paper presents a cDFA based regular counting constraint with explanations. To this end we explain the explanations for propagating the count and variables in the input sequence and provide an exponential time algorithm for inference checking. In addition, we analyze the performance of our constraint compared to two DFA-based regular constraints propagated respectively with decomposition and Gvosdiovas et al.'s[5] LCG implementation of Pesant's[6] propagator to show that it is beneficial to use our propagator when [Insert properties of cDFA]. We show why nonograms are a bad benchmark for this purpose and that generation of random cDFA's and equivalent DFA's is a good one.

[Insert additional relevant results]

Section 2 provides the background knowledge our research was based on, explaining CP and LCG, defining the used finite automata and explaining Beldiceanu et al.’s algorithm. Section 3 discusses the history of the field of CP and how it relates to our contributions. Section 4 presents how we provide explanations for LCG and our approach to benchmarking. Section 5 elaborates on how the experiments were run and showcases the compiled data. Section 6 discusses considerations with regards to ethics and the reproducibility of our research. Section 7 summarizes our contributions, highlighting unexplored avenues of research.

2 Preliminaries

This section explains the following topics relevant to our research: constraint programming in Section 2.1, lazy clause generation in Section 2.2, DFA and cDFA in Section 2.3 and the cDFA-based constraint in Section 2.4.

2.1 Constraint Programming

Constraint programming (CP) is a paradigm in which combinatorial problems are modeled with decision variables and constraints that need to be satisfied.

In this field, finite domain (FD) CP is one of the biggest subdomains. FD constraint solvers are programs that can solve FD combinatorial **satisfaction** and **optimization** problems, which respectively ask for *any* solution and a solution that *optimizes* a given metric. FD constraint solvers are able to prune branches of the search space without solutions, without entering them. They do this by deducing which assignments of the variables prevent the constraints from being satisfied, or in other terms results in a conflict. This can often lead to a sizable decrease in the search space compared to a brute-force approach.

The algorithms that make such deductions are called **propagators**, pruning based on a propagator’s deduction **propagating** a constraint, and a single results of propagating a **propagation**. A proper propagator should never prune solutions.

There are two approaches to propagating a constraint. One is to use an algorithm designed for that specific constraint. The other is to decompose the constraint into many *simpler* constraints with *simpler* propagators. When deciding which algorithm or decomposition to use, it is worth considering the trade-off between strength—*how many* propagations are applied and *how useful* are they—and speed—*how fast* are propagations computed.

Two relevant concepts that apply to propagators are **bounds consistency** and **domain consistency**. A propagator is **bounds consistent** when, if you apply it sequentially, it doesn’t propagate on the *bounds* of the domain more than once; It has made all the deductions it can regarding the bounds. A propagator is **domain consistent** when, if you apply it sequentially, it doesn’t propagate on the domain more than once; It has made all the deductions it can.

2.2 Lazy Clause Generation

Lazy clause generation (LCG) is an extension to FD constraint solvers based on solvers from the subdomain of Boolean CP: satisfiability-solvers (SAT-solvers). SAT-solvers are able to solve Boolean satisfiability problems; Given a set of variables and a set of Boolean clauses, they find an assignments of the variables that makes all clauses true. SAT-solvers

are able to make use of **conflict learning**, something FD constraint solvers can not do. When a conflict occurs, they can deduce which decisions resulted in that conflict from the clauses in an implication graph. The negation of the clause that represents that decision—also referred to as a **nogood**—is then added as a new constraint. This is especially useful for searching large domains.

LCG adds advantages of SAT-solvers—like **conflict learning**—to FD constraint solvers. It does this by making propagations provide Boolean clauses for both the reason for propagation and the propagation itself. This combination of reason and propagation is called an **explanation**. Generally, the *shorter* the **explanation**, the more useful it is, as the more clauses you add to the reason, the less generalizable it is to the rest of the search space. Because of this, LCG is known to often learn more from decomposition into smaller constraints, compared to more complex propagators[7].

To ensure the correctness of **explanations**, it is good practice to implement an **inference checker** for a propagation. When solving a problem with an **inference checker** enabled, it will verify each **explanation**. It does this by applying each clause of an **explanation**'s reason and the negation of its propagation, after which the constraint should no longer be satisfied if the **explanation** was correct. Because this introduces overhead, **inference checkers** are generally not enabled by default.

2.3 Finite Automata

2.3.1 DFA

Recall that a deterministic finite automaton (DFA) is a finite automaton that recognizes regular languages. It is defined by the five-tuple[8]:

$$\langle Q, \Sigma, \delta, q_0, F \rangle$$

Where:

1. Q is a finite set called the *states*;
2. Σ is a finite set called the *alphabet*;
3. $\delta : Q \times \Sigma \rightarrow Q$ is the *transition function*;
4. $q_0 \in Q$ is the *initial state*;
5. $F \subseteq Q$ is the *set of accept states*.

Given a finite sequence X , where for each $x_i \in X : x_i \in \Sigma$, a DFA accepts X iff it is in an accepting state after consuming X . That is: starting in q_0 ($q \leftarrow q_0$), for each $x_i \in X$ in order, go from one state to the next, according to δ ;

$$q' = \delta(q, x_i) : q \leftarrow q'$$

2.3.2 cDFA

Beldiceanu et al.[3] describes a subclass of counter-DFA's (cDFA). When referring to cDFA's in this paper, we mean this specific subclass. It is similar to a DFA, but differs in two aspects: It only has accepting states; It has a singular counter that is initialized to zero and increments on certain transitions. In this paper it is defined by the five-tuple:

$$\langle Q, \Sigma, \delta, q_0, K \rangle$$

Where:

1. Q is a finite set called the *states*;
2. Σ is a finite set called the *alphabet*;

3. $\delta : Q \times \Sigma \rightarrow Q \times \mathbb{N}$ is the *transition function*;
4. $q_0 \in Q$ is the *initial state*;
5. $K \subseteq \mathbb{N}$ is the *set of accepted counts*.

Given a finite sequence X , where for each $x_i \in X$ s.t. $x_i \in \Sigma$, a DFA accepts X iff its counter has an accepting count after consuming X . That is: starting in q_0 ($q \leftarrow q_0$), with counter k initialized to zero ($k \leftarrow 0$), for each $x_i \in X$ in order, go from one state to the next and increment k with the count of the transition, according to δ ;

$$\langle q', inc \rangle = \delta(q, x_i) : q \leftarrow q', k \leftarrow k + inc$$

For ease of notation, when $\delta(q, \ell) = \langle q', inc \rangle : \delta_Q(q, \ell) = q'$ and $\delta_{\mathbb{N}}(q, \ell) = inc$

2.4 cDFA based constraint

2.4.1 Definition

The cDFA-based regular counting constraint is defined as:

$$\text{regular_cDFA}(X, A, count)$$

where:

- $dom(var)$ is the domain of variable var ;
- X is a sequence of variables $x_1 \dots x_n$, $dom(x_i) \subseteq \Sigma$;
- A is a cDFA;
- $count$ is a variable, with $dom(count) \subseteq \mathbb{N}$;

The constraint is satisfied when A counts to $count$ when consuming sequence X .

2.4.2 Propagating the cDFA based constraint

To explain the propagation proposed by [3] we need to start by defining the following functions:

- $\text{QCF}(i)$: The set of pairs $\langle q, c \rangle$, where c is the set of possible counter increases when the cDFA consumes a sequence of length i , starting in the starting state and ending in state q —the possible counter increases c to reach state q in i transitions.
- $\text{QCB}(i)$: The set of pairs $\langle q, c \rangle$, where c is the set of possible counter increases after visiting state q after exactly $n - i$ transitions when the cDFA consumes a sequence of length n , starting in the starting state and ending in some other state—the possible counter increases c to reach a final state from state q in i transitions.
- $\underline{\text{QCF}}(i)$ (and $\overline{\text{QCF}}(i)$): The set of pairs $\langle q, c \rangle$, where c is the minimum (maximum) counter increase to reach state q .
- $\underline{\text{QCB}}(i)$ (and $\overline{\text{QCB}}(i)$): The set of pairs $\langle q, c \rangle$, where c is the minimum (maximum) counter increase to reach a final state from state q .

Using the following two functions to respectively keep only the tuples with the minimum and maximum cost for a given state:

$$\text{trimMin}(S) = \{\langle q, c \rangle \in S \mid \nexists \langle q, c' \rangle \in S : c' < c\}$$

$$\text{trimMax}(S) = \{\langle q, c \rangle \in S \mid \nexists \langle q, c' \rangle \in S : c' > c\}$$

$\underline{\text{QCF}}(i)$, $\overline{\text{QCF}}(i)$, $\underline{\text{QCB}}(i)$ and $\overline{\text{QCB}}(i)$ can be implemented inductively:

$$\begin{aligned}
\underline{\text{QCF}}(i) &= \begin{cases} \{\langle q_0, 0 \rangle\} & \text{if } i = 0 \\ \text{trimMin}(\{\langle \delta_Q(q, \ell), c + \delta_{\mathbb{N}}(q, \ell) \rangle \mid \langle q, c \rangle \in \underline{\text{QCF}}(i-1), \ell \in \text{dom}(x_i)\}) & \text{if } i \in [1, n] \end{cases} \\
\overline{\text{QCF}}(i) &= \begin{cases} \{\langle q_0, 0 \rangle\} & \text{if } i = 0 \\ \text{trimMax}(\{\langle \delta_Q(q, \ell), c + \delta_{\mathbb{N}}(q, \ell) \rangle \mid \langle q, c \rangle \in \overline{\text{QCF}}(i-1), \ell \in \text{dom}(x_i)\}) & \text{if } i \in [1, n] \end{cases} \\
\underline{\text{QCB}}(i) &= \begin{cases} \{\langle q, 0 \rangle \mid \exists c \in \mathbb{N} : \langle q, c \rangle \in \underline{\text{QCB}}(i)\} & \text{if } i = n+1 \\ \text{trimMin}(\{\langle q, c' + inc \rangle \mid \langle q', c' \rangle \in \underline{\text{QCB}}(i+1), \ell \in \text{dom}(x_i), \delta(q, \ell) = \langle q', inc \rangle\}) & \text{if } i \in [1, n] \end{cases} \\
\overline{\text{QCB}}(i) &= \begin{cases} \{\langle q, 0 \rangle \mid \exists c \in \mathbb{N} : \langle q, c \rangle \in \overline{\text{QCB}}(i)\} & \text{if } i = n+1 \\ \text{trimMax}(\{\langle q, c' + inc \rangle \mid \langle q', c' \rangle \in \overline{\text{QCB}}(i+1), \ell \in \text{dom}(x_i), \delta(q, \ell) = \langle q', inc \rangle\}) & \text{if } i \in [1, n] \end{cases}
\end{aligned}$$

Propagation on the bounds of *count* can then be implemented as:

$$\text{dom}(\text{count}) \geq \min(c) \mid \langle q, c \rangle \in \underline{\text{QCF}}(n)$$

$$\text{dom}(\text{count}) \leq \max(c) \mid \langle q, c \rangle \in \overline{\text{QCF}}(n)$$

To implement propagation for variables in X , a value ℓ is removed from the domain of x_i iff:

$$\forall q \left[\begin{array}{l} \langle q, \underline{c} \rangle \in \underline{\text{QCF}}(i-1) \\ \langle q, \bar{c} \rangle \in \overline{\text{QCF}}(i-1) \\ \langle q', inc \rangle = \delta(q, \ell) \\ \langle q', \underline{c}' \rangle \in \underline{\text{QCB}}(i+1) \\ \langle q', \bar{c}' \rangle \in \overline{\text{QCB}}(i+1) \end{array} \right] : [\underline{c} + inc + \underline{c}', \bar{c} + inc + \bar{c}'] \cap \text{dom}(\text{Count}) = \emptyset$$

3 Related work

Constraint solving was originally proposed by Lauriere[9], as an approach to solve combinatorial problems. Later, Jaffar and Lassez[10] coined constraint logic programming as an extension. One of its capabilities was finite domain (FD) constraint solving. It improved the ease of modeling combinatorial problems.

Lazy clause generation (LCG) was proposed by Ohrimenko et al.[11]. It implemented FD propagation in a SAT-solver, effectively combining the advantages of FD constraint solving and SAT-solving. LCG was then re-engineered by Feydy and Stuckey[7], by implementing a SAT-solver as a propagator inside a FD solver and having other propagators generate explanations to populate it with. It proved more efficient for large search spaces and boosted the performance of decomposition into global constraints, as FD constraint solvers extended with LCG can make use of conflict-learning.

The DFA based regular constraint and an accompanying domain consistent propagator were originally proposed by Pesant[6]. Gvosdiovas et al.[5] recently created a version of Pesant's propagator with explanations for LCG and extended it to work with non-deterministic finite automata (NFA). As we also extend a propagator with explanations for LCG, this research was influential in how we approached our research. A comparison between their NFA-based constraint and our cDFA-based constraint does not seem useful to us, as we can't think of ways that non-determinism provides significant advantages for regular counting.

Using cDFA's for regular counting was originally proposed by Beldiceanu et al.[4], as the use of a counter allows for a simpler representation to count regular patterns opposed to normal DFA's. Later, Beldiceanu et al.[3] proposed propagators for "at least", "at

most” and “exact” counting of regular patterns using cDFA’s, that are domain consistent on the input sequence. Their “at most” and “at least” propagators were shown to also be domain consistent on the count, whereas their “exact” propagator only provides bounds consistency. They further proved that computing satisfiability for their “exact” constraint is NP-Hard.

Although they show that their propagators prune more than decomposition of a DFA-based regular constraint, they only show this for FD constraint solving *without* LCG. This poses the question if that is still the case when implemented *with* LCG, which can benefit from decomposition. Another question left unanswered is for what sizes of cDFA it offers an significant advantage to model problems with them. Our research answers these questions.

Martin and Pearson[12] provided an algorithm that decides when bounds consistency implies domain consistency for a given cDFA. This also analyzes when cDFA’s are more efficient, but does so by analyzing their structure in stead of their size and provides no comparison with DFA’s.

The cost-regular constraint proposed by Demassey et al.[13] has many similarities—both accumulate values when taking transitions in finite automata and compare the accumulation to a variable—to the regular counting constraint proposed by Beldiceanu et al.[3], but propagates less and has asymptotically worse space complexity. No comparison of these two propagators extended with explanations for LCG has also not been done as well, but due to the limited scope of our research, we left this untouched.

4 Propagating regular_cDFA with explanations

This section showcases our contributions. Section 4.1 explains our explanations for propagating and the accompanying inference checker, and Section 4.2 describes the approach to compare the constraints.

4.1 Explanations

These explanations are directly based on the propagations from Beldiceanu et al.[3]. As such, they are trivial, but lay the groundwork for future improvements.

4.1.1 The count

The propagation clause for *count* is always an update to the bounds of the domain:

$$[count \geq value] \text{ or } [count \leq value]$$

The clauses making up the reason for propagating on *count* represent the domains of the variables in X . All $c \in dom(count)$ are independent from each other.

4.1.2 The input sequence

The propagation clause for the variables in X is an inequality clause:

$$[x_i \neq value]$$

The clauses making up the reason for propagating on x_i represent the domains of the variables in X and the set of clauses for *count* that describe

$$\bigcup \left\{ dom(count) \cap [\underline{c} + inc + \underline{c}', \bar{c} + inc + \bar{c}'] \mid \forall q \left[\begin{array}{l} \langle q, \underline{c} \rangle \in \underline{QCF}(i-1) \\ \langle q, \bar{c} \rangle \in \overline{QCF}(i-1) \\ \langle q', inc \rangle = \delta(q, \ell) \\ \langle q', \underline{c}' \rangle \in \underline{QCB}(i+1) \\ \langle q', \bar{c}' \rangle \in \overline{QCB}(i+1) \end{array} \right] \right\}$$

4.1.3 Inference checker

When implementing an inference checker for $\text{regular_cDFA}(X, A, \text{count})$, there is one unfortunate fact; Computing satisfaction for regular_cDFA is an NP-Hard problem, as Beldiceanu et al.[3] have proven by reduction from SAT. This means that it is hard to calculate if the constraint is still satisfied. We have nonetheless implemented an algorithm and were able to verify our explanation for small instances using it. Bigger instances quickly lead to the program not completing.

Our inference checker first computes the set of all possible counts after consuming X , $\text{QCF}(n)$, given the explanation. It then checks if there exists a $c \in \text{dom}(\text{count})$, such that $c \in \text{QCF}(n)$. If that is the case, the constraint is still satisfied.

4.2 Benchmarks

In an attempt to show when our cDFA-based propagator offers advantages over DFA based ones, we considered two benchmarks. Our initial plan was based on modelling nonograms, but we found that that does not exploit advantages of cDFA-based propagation meaningfully. We then pivoted to generating random cDFA's and equivalent DFA's.

4.2.1 Nonograms

Nonograms are well-known puzzles. They consist of an empty grid, where each row and column has a “hint”—a list of numbers—that corresponds to what squares in the row or column can be filled. Each number n in the hint corresponds to a subsequence of n filled in squares; The subsequences occur in order; Subsequences are separated by at least one empty square.

As this problem relies on patterns that count, it seems like a good fit to model with cDFA's. And yes, it is possible to model the hints for the rows and columns with regular counting; a given hint $[k, m, n]$ can be modeled by the regular expression “ $0^*1\{k\}0^*1\{m\}0^*1\{n\}0^*$ ”. There are two problems when modeling with cDFA's however.

First of all, the count of a subsequence is always a fixed value, whereas regular_cDFA is able to propagate on the count. This means modeling nonograms does not exploit an advantage of cDFA.

Second of all, there are multiple patterns that are counted. This poses the biggest problem. cDFA's only have one counter. As such, nonograms can not be trivially modeled using cDFA's in a way that results in a smaller encoding than when modeled with a DFA.

There is one approach however. If a cDFA increments by at most one, the maximum count after consuming a sequence of n values is n . To implement a second counter, whose count can be distinguished from the first, edges incrementing by $n + 1$ can thus be used. An accepting count can then be encoded as:

$$1 \cdot \text{count}_1 + (n + 1) \cdot \text{count}_2$$

This can be generalized for k counters as:

$$\sum_{i=1}^k (n + 1)^{i-1} \cdot \text{count}_i$$

Note that this essentially represents each count as a digit in a base $n + 1$ number.

[Insert image of trivial cDFA, encoded cDFA and DFA models describing the same hint]

This introduces a new problem however: The accepting count grows exponentially proportional to the amount of encoded counts. For small nonograms, this is no issue, as they have small sequences and consequently a small amount of numbers per hint. As

nonograms grow in size however, so do their sequences and consequently the amount of numbers per hint. For a 30x30 nonogram, that thus has sequences of length 30, it is not uncommon to see hints with seven or more numbers. As $31^7 > 2^{32}$, the accepting counts used to model larger nonograms are unable to be represented by either signed or unsigned 32-bit integers.

These two factors—having a single count per sequence and exponential blowup of the accepted count—make it impractical to pursue this problem as a benchmark.

4.2.2 Random equivalent constraints

We decided to use randomly generated equivalent DFA and cDFA to benchmark instead. This tackles both issues with the previous approach by generating finite automata that count only one pattern and have multiple accepting counts. We achieve this in the following three steps:

1. We first construct a random incomplete intermediate DFA.
2. We convert the intermediate DFA to a cDFA.
3. We use the incomplete DFA to construct a DFA that is equivalent to the generated cDFA, that is then minimized.

To work with the DFA and to minimize the equivalent DFA, the python library FAdo¹ was used.

4.2.3 Constructing the intermediate DFA

A DFA with $n + 1$ states and only one accepting state—where n is the amount of states our resulting cDFA will end up with—is generated randomly. This is done in a way that a random Hamiltonian path from the starting state to the accept state exists, to guarantee that the resulting automata accept some sequences. The accept state is incomplete; It does not have any outgoing edges.

[Insert image of an example intermediate DFA]

4.2.4 Constructing the cDFA

The ingoing edged to the accept state of the intermediate DFA are marked as incrementing by one. The accept state is merged with the starting state. All states are made accepting.

[Insert image of cDFA generated from example intermediate DFA]

4.2.5 Constructing the equivalent DFA

From the intermediate DFA $\max(counts) + 1$ copies are made.

$$\{copy_i \mid i \in 0, 1, \dots, \max(counts)\}$$

Each accepting state of $copy_i$ is merged with the starting state of $copy_{i+1}$, essentially concatenating the pattern $\max(counts) + 1$ times. The final incomplete state is made into a rejecting trap-state. States corresponding to the rejecting states of $copy_i$ are accepting iff $i \in counts$.

[Insert images of cDFA being created by first concatenating the copies and then setting the accepting states.]

¹<https://fado.dcc.fc.up.pt/>

5 Experimental Setup and Results

Our results show that for cDFA's with [insert qualities of cDFA's], it is more effective to use the regular_cDFA constraint with our propagator than a regular constraint with Gvosdiosvas et al.'s[5] propagator.

Our results show that for cDFA's with [insert qualities of cDFA's], it is more effective to use the regular_cDFA constraint with our propagator than a decomposed regular constraint with Gvosdiosvas.

[Insert additional relevant result summary]

5.1 Setup

The regular_cDFA constraint and the DFA-based regular constraint were implemented in Pumpkin[14] using Rust. The DFA-based regular constraint was adapted from the code of Gvosdiosvas et al.[5]. The decomposition of the DFA-based regular constraint uses the decomposition built into Pumpkin. Minizinc[insert reference or footnote] was used to model our benchmarks using the constraints. Those models were then provided with json files describing individual instances to run, which were generated by Python scripts. Shell scripts were used to generate many instances at once. A python script was used to run all instances of a benchmark and store the results. A python script was used to aggregate, analyse and graph the data.

This approach was inspired by Gvosdivas et al.. An important change is that we opted to not parallelize the running of the instances of our benchmarks, to make sure that their solve speed was impacted as little as possible by random variables.

The benchmarks were run on a laptop with [Insert specs] running NixOS without a graphical user interface, to minimize background processes.

5.2 Results

[Show box plots containing amount of states of the finite automata, amount of propagations, solve speed and the number of conflicts. Grouped by cDFA, Pesant DFA and decomposed DFA. Also group by parameters such as amount of states, alphabet, etc.]

6 Responsible Research

6.1 Data generation

The data we used for benchmarking was all synthetically generated by scripts. As such, its usage is unrestricted by ethical considerations. One important detail to mention however, is that FAdo¹—a python library for manipulating finite automata—falls under the GNU General Public License. As such, the python script using it to generate a random equivalent DFA and cDFA also falls under this license. The rest of our contributions fall under the MIT license.

6.2 AI usage

LLM's—and other generative AI—have become prevalent not only in everyday life, but also in academia. However, we have opted to not use any generative AI during the course of this research, as it makes it easy to outsource thinking, which can lead to research that is less critical. This paper and our contributions have been human generated.

6.3 Reproducibility

All our experiments and code have been version controlled since the beginning using git, and are available in our *experiment suite*². This was done not only to allow others to check our work, but also in an effort to make any material that might be of interest for future research readily available. Our experiment suite includes the following:

- A fork of Pumpkin, extended with the constraints used;
- Scripts to generate and run the benchmarks;
- Results of the benchmarks;
- A script to process and visualise the results.

In addition to this, all randomly generated instances were seeded and labeled with the used seed.

By using *devenv*³ we have tried to prevent additional setup and dependency conflicts that might otherwise have occurred when running the experiments on other machines.

7 Conclusion

In this paper we have presented the first ever extension with explanations for lazy clause generation (LCG) of Beldiceanu et al.’s[3] propagator for the exact regular counting constraint. We have shown that when [insert conditions of cDFA], our propagator provides an advantage over an equivalent DFA based regular constraint propagated with Gvosdiovas et al.’s[5] extension with explanations for lcg of Pesant’s[6] propagator, and that when [insert conditions of cDFA], it provides an advantage over an equivalent DFA based regular constraint decomposed into global constraints. This informs researchers on how to solve finite domain combinatorial problems involving regular counting more effectively. [insert additional important take-aways].

In addition, we have documented two ways we have attempted to compare the aforementioned constraints. For comparing cDFA’s with DFA’s, we have shown the shortcomings of modeling nonograms with cDFA’s and the effectiveness of generating random equivalent finite automata.

7.1 Future Work

There are multiple opportunities for future research that we would like to see explored:

- As LCG is known to learn more from simpler propagation[7], how does decomposing exact regular counting into at-most and at-least regular counting[3]—extended with explanations for LCG—compare to our propagator?⁵
- Similarly, how can cDFA based regular counting be decomposed into global constraints? How does this compare to other constraints?
- how does our propagator compare to the similar, but more widely used `cost_regular` constraint—decomposed or propagated with an extension to Demassey et al.’s[13] propagator—with explanations for LCG?

²<https://github.com/ICorvidusI/experiment-suite-RP-2026>

³<https://devenv.sh/>, a tool for declarative development environments that isolates and version locks packages, based on the *nix package manager*⁴

⁴<https://nixos.org/>

⁵A non-verified propagation for the at-most regular counting constraint can be found—commented out—in the methods of our `regular_cdfa` propagator.²

- Can Martin and Pearson’s[12] algorithm—to compute when bounds consistency implies domain consistency for cDFA-based regular counting—be used to improve our propagator’s speed or explanations?
- As our explanations for LCG are trivially based on the propagation, can they be shortened to provide better conflict-learning?
- Can our inference checker be optimized in any way? One approach we could see working is to prune counts of $\text{QCF}(i)$ —see Section 2.4—that exceed the upper bound of the domain of *count*.

References

- [1] C. Chapman and K. T. Stolee, “Exploring regular expression usage and context in Python,” in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, in ISSTA 2016. New York, NY, USA: Association for Computing Machinery, July 2016, pp. 282–293. doi: 10.1145/2931037.2931073.
- [2] D. Hu and Z. Wu, “An efficient string solver for string constraints with regex-counting and string-length,” *Journal of Systems Architecture*, vol. 160, p. 103340, Mar. 2025, doi: 10.1016/j.sysarc.2025.103340.
- [3] N. Beldiceanu, P. Flener, J. Pearson, and P. V. Hentenryck, “Propagating Regular Counting Constraints.” Accessed: Apr. 29, 2026. [Online]. Available: <http://arxiv.org/abs/1309.7145>
- [4] N. Beldiceanu, M. Carlsson, and T. Petit, “Deriving Filtering Algorithms from Constraint Checkers,” in *Principles and Practice of Constraint Programming – CP 2004*, M. Wallace, Ed., Berlin, Heidelberg: Springer, 2004, pp. 107–122. doi: 10.1007/978-3-540-30201-8_11.
- [5] J. Gvozdiavas, D. Emir, and F. Maarten, “NFA propagator for regular constraint with explanations,” Jan. 2025, Accessed: Apr. 21, 2026. [Online]. Available: <https://repository.tudelft.nl/record/uuid:16153696-2b90-46f7-8b48-03455d427ba1>
- [6] G. Pesant, “A Regular Language Membership Constraint for Finite Sequences of Variables,” in *Principles and Practice of Constraint Programming – CP 2004*, M. Wallace, Ed., Berlin, Heidelberg: Springer, 2004, pp. 482–495. doi: 10.1007/978-3-540-30201-8_36.
- [7] T. Feydy and P. J. Stuckey, “Lazy Clause Generation Reengineered,” in *Principles and Practice of Constraint Programming - CP 2009*, I. P. Gent, Ed., Berlin, Heidelberg: Springer, 2009, pp. 352–366. doi: 10.1007/978-3-642-04244-7_29.
- [8] M. Sipser, “Formal definition of a finite automaton,” *Introduction to the Theory of Computation*. Cengage Learning, pp. 35–40, 2013.
- [9] J.-L. Lauriere, “A language and a program for stating and solving combinatorial problems,” *Artificial Intelligence*, vol. 10, no. 1, pp. 29–127, Feb. 1978, doi: 10.1016/0004-3702(78)90029-2.
- [10] J. Jaffar and J.-L. Lassez, “Constraint logic programming,” in *Proceedings of the 14th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, in POPL '87. New York, NY, USA: Association for Computing Machinery, Oct. 1987, pp. 111–119. doi: 10.1145/41625.41635.
- [11] O. Ohrimenko, P. J. Stuckey, and M. Codish, “Propagation = Lazy Clause Generation,” in *Principles and Practice of Constraint Programming – CP*

- 2007, C. Bessière, Ed., Berlin, Heidelberg: Springer, 2007, pp. 544–558. doi: 10.1007/978-3-540-74970-7_39.
- [12] B. Martin and J. Pearson, “When bounds consistency implies domain consistency for regular counting constraints,” *Constraints*, vol. 27, no. 3, pp. 161–167, July 2022, doi: 10.1007/s10601-022-09333-0.
 - [13] S. Demasse, G. Pesant, and L.-M. Rousseau, “A cost-regular based hybrid column generation approach,” in *Constraints*, 2006, pp. 315–333. doi: 10.1007/s10601-006-9003-7.
 - [14] M. Flippo, K. Sidorov, I. Marijnissen, J. Smits, and E. Demirović, “A Multi-Stage Proof Logging Framework to Certify the Correctness of CP Solvers,” in *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, P. Shaw, Ed., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 307. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 11:1–11:20. doi: 10.4230/LIPIcs.CP.2024.11.